Advanced Jetpack Compose

Intents, Layouts, Intrinsic Measurements & Concurrency

Muhammad Naveed

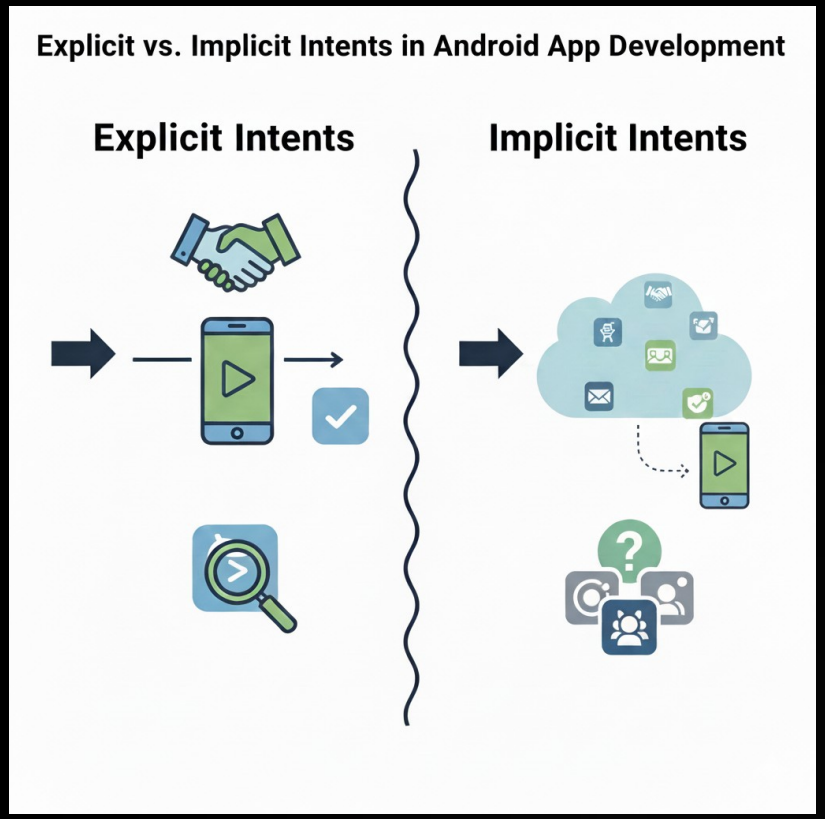
November 26, 2025

Intents & Navigation

The glue between Android components

What are Intents?

- **Definition:** An abstract description of an operation to be performed. They act as messages between components (Activities, Services, Broadcast Receivers).
- **Explicit Intents:** Specify the exact component (class) to start. Used for internal navigation (e.g., ActivityA -> ActivityB).
- **Implicit Intents:** Declare a general action (e.g., "Open URL") and let the Android system find a suitable app to handle it.



Intent Structure & Data

Anatomy

Action: The general action to perform (e.g., `ACTION_VIEW`).

Data: The URI of data to operate on.

Category: Additional info about the component kind.

Transferring Data

Data is passed via **Extras**, which is essentially a Bundle of key-value pairs.

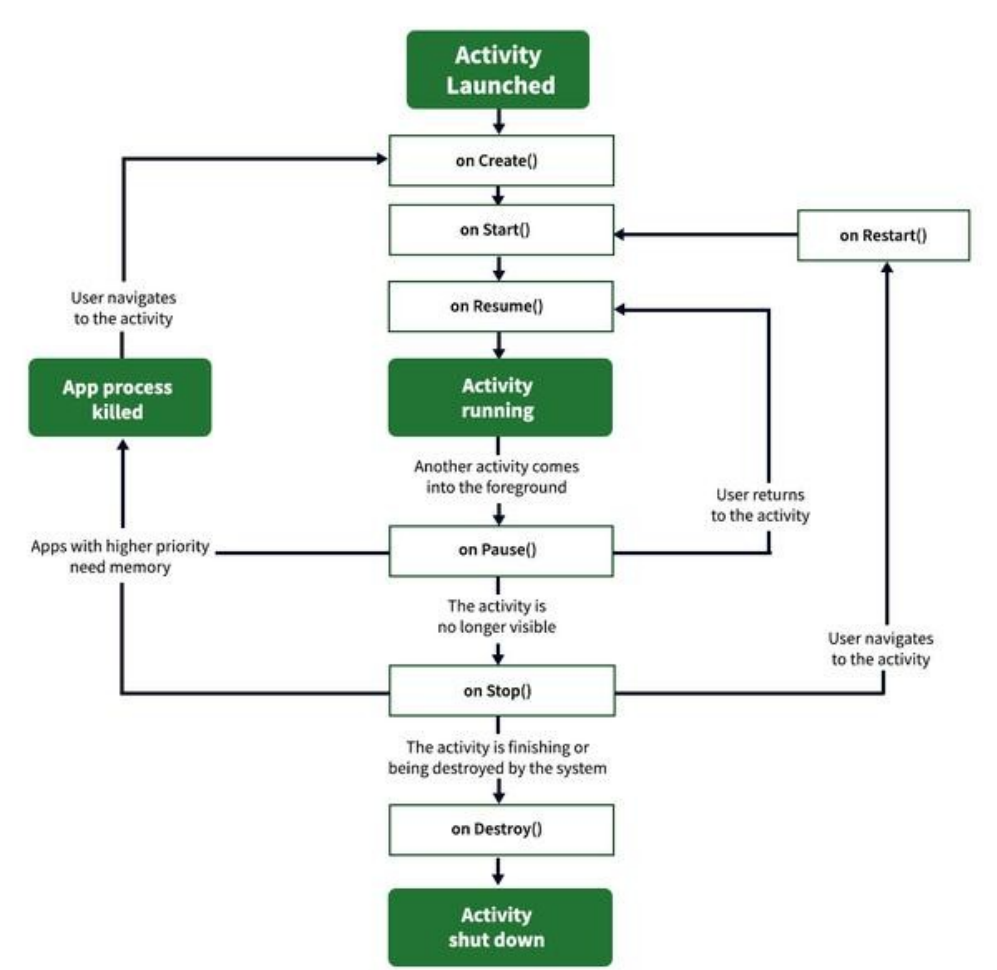
```
intent.putExtra("USER_ID", 123)

val id = intent.getIntExtra("USER_ID")
```

Intents & Lifecycle

The Transition Flow

- When Activity A starts Activity B using an Intent, they undergo a specific lifecycle sequence.
- **Activity A:** onPause() → onStop() (if no longer visible).
- **Activity B:** onCreate() → onStart() → onResume().
- When the user presses Back, Activity B is destroyed, and Activity A restarts and resumes.



Implicit Intent Resolution

Intent Filters

Declared in AndroidManifest.xml. They specify the types of intents an Activity, Service, or Broadcast Receiver can respond to.

System Matching

The Android system matches the Intent's Action, Data, and Category against all installed filters to find a candidate.

App Chooser

If multiple apps match the intent (e.g., multiple browsers), the system displays a dialog for the user to pick their preferred app.

Mastering Navigation

A comprehensive guide to building modern, type-safe navigation flows in Android.

The Paradigm Shift (Official Link)

Single Activity

Gone are the days of multiple Activities and FragmentManagers. In Compose, your app typically resides in a single Activity.

Gone are the days of multiple Activities and FragmentManagers. In Compose, your app typically resides in a single Activity.

Composable Destinations

Destinations are no longer Fragments or Layout XMLs. They are pure Kotlin @Composable functions.

This allows for a smoother, state-driven UI transition and simpler lifecycle management.

Setup & Dependencies

Gradle Configuration

- To get started, you need to include the Navigation Compose artifact in your build.gradle.kts.
- For Type-Safe navigation (the modern standard), we also include the Kotlin Serialization plugin.

```
dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform( dependencyProvider = libs.androidx.compose.bom))
    implementation(libs.androidx.compose.ui)
    implementation(libs.androidx.compose.ui.graphics)
    implementation(libs.androidx.compose.ui.tooling.preview)
    implementation(libs.androidx.compose.material3)
    implementation("androidx.navigation:navigation-compose:2.9.6")
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform( dependencyProvider = libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.compose.ui.test.junit4)
    debugImplementation(libs.androidx.compose.ui.tooling)
    debugImplementation(libs.androidx.compose.ui.test.manifest)
}
```

The Three Pillars

NavController

The central API for the navigation state. It keeps track of the back stack of Composables and orchestrates navigating between them.

NavHost

The container Composable that displays the current destination. It links the NavController to a navigation graph.

NavGraph

A data structure that defines all possible routes (destinations) in your application and how they connect to each other.

Defining Routes

Type Safety First

- Modern Compose Navigation uses @Serializable objects or classes to define routes.
- This replaces the old error-prone string-based routes (e.g., "profile/{id}"), ensuring compile-time safety for your arguments.

```kotlin
import kotlinx.serialization.Serializable

// Define destinations as Serializable types
@Serializable
object Home

@Serializable
data class Profile(
    val id: String,
    val showDetails: Boolean = false
)
```

Building the Graph

NavHost Composable

- navController: The controller instance created via rememberNavController().
- startDestination: The object representing the first screen to show.
- composable: A builder function to add a destination to the graph. The type T is your serializable route class.

```
val navController = rememberNavController ()

NavHost (
    navController = navController,
    startDestination = Home
) {
    composable <Home> {
        HomeScreen (
            onNavigateToProfile = { id →
                navController.navigate (Profile (id))
            }
        )
    }

    composable <Profile> {
        ProfileScreen ()
    }
}
```

Navigation Actions

Navigate

```
navController.navigate(
    Profile(id="123")
)
```

Pushes a new destination onto the back stack. State is preserved automatically.

Pop Back Stack

```
navController.popBackStack()
```

Removes the current destination and returns to the previous one. Returns false if the stack is empty.

Pop Up To

```
navController.navigate(Home) { popUpTo { inclusive = true } }
```

Useful for login flows to prevent users from going back to the login screen.

Deep Links

Configuring Deep Links

- Deep links allow users to navigate directly to a specific screen from outside the app (e.g., a web link or notification).
- They are defined within the composable builder using the deepLinks parameter.

```
composable<Profile>(
    deepLinks = listOf(
        navDeepLink<Profile>(
            basePath = "www.example.com/profile"
        )
    )
) {
    // ... content
}
```

* Requires AndroidManifest intent-filter configuration.

Nested Navigation

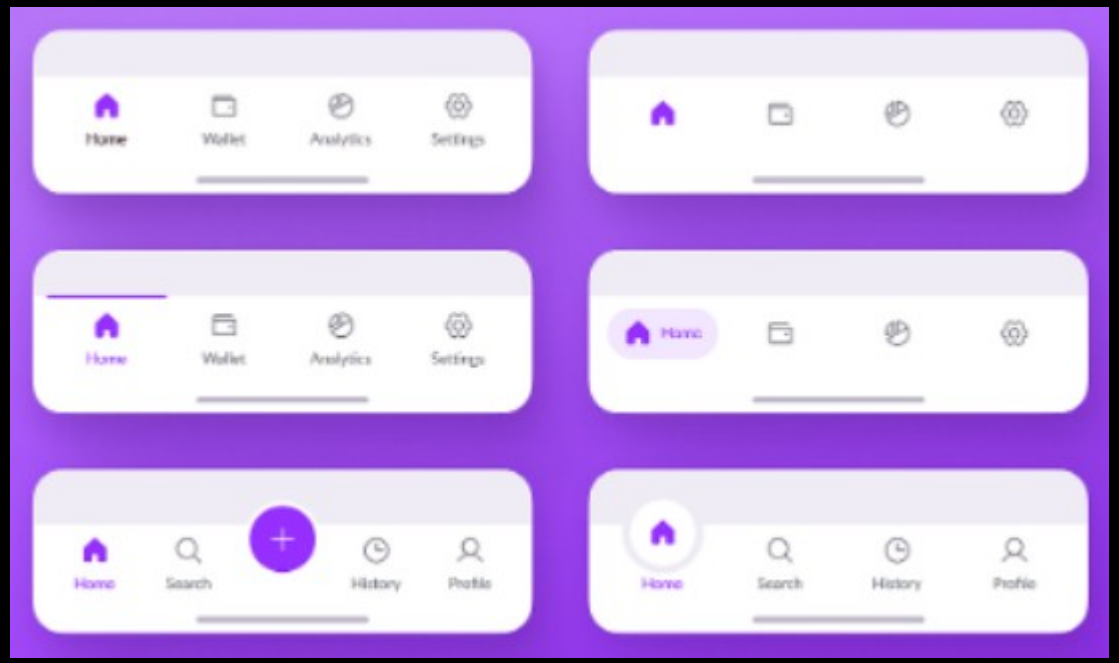
Organizing Large Graphs

- As your app grows, putting all routes in a single list becomes unmanageable.
- Nested Graphs allow you to modularize your navigation logic. For example, a "Login Flow" or "Checkout Flow" can be its own self-contained graph.
- Use the navigation builder to create a sub-graph.

UI Integration

Bottom Navigation Bar

- Integrating with Scaffold and BottomNavigation is straightforward.
  - Observe the currentBackStackEntry via navController.
  - Check if the current destination's hierarchy matches the tab's route.
  - Navigate using popUpTo(startDestination) and launchSingleTop to avoid stack buildup.

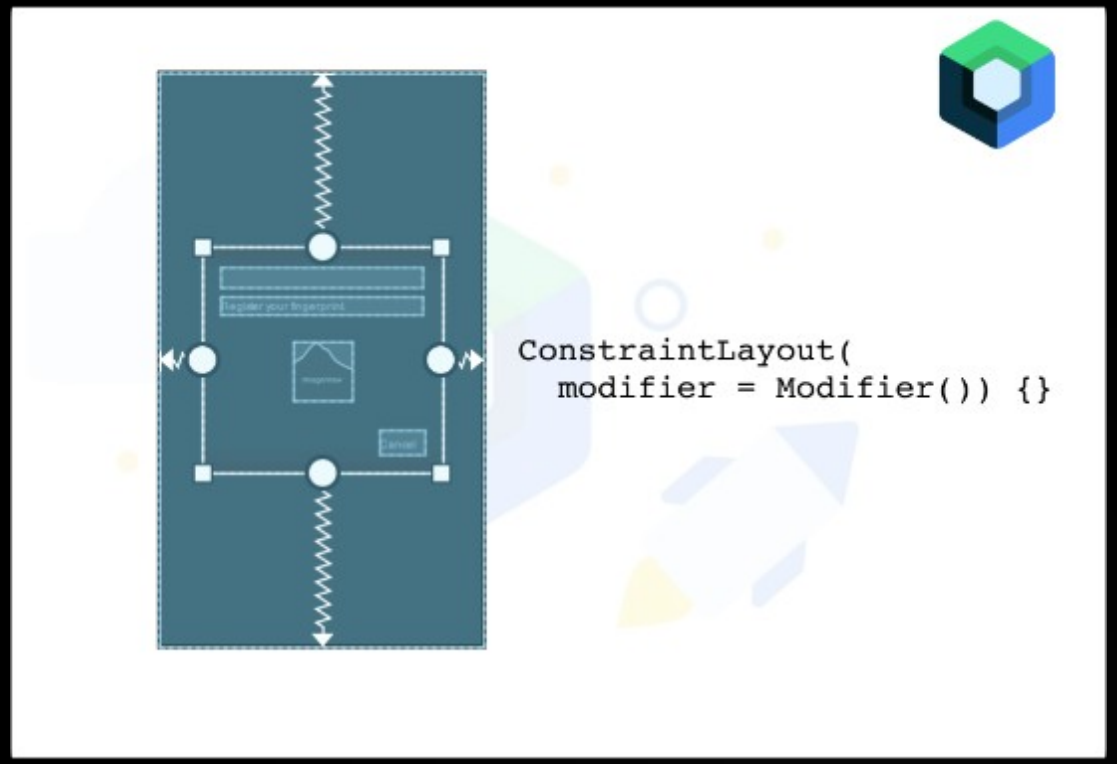


ConstraintLayout in Compose

Mastering complex UI logic with flat hierarchies

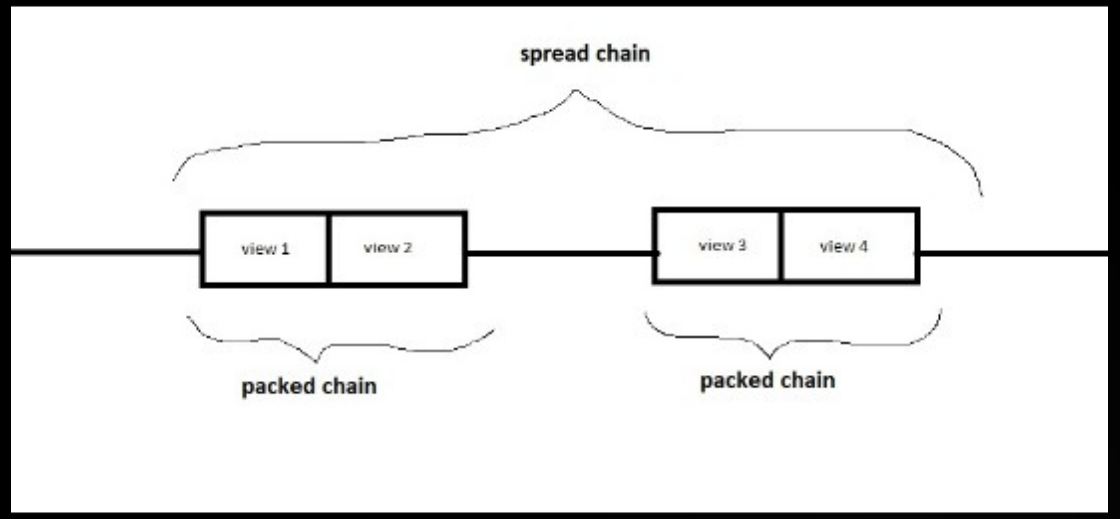
ConstraintLayout Basics

- **Constraints:** Rules that define a composable's position relative to others or the parent.
- **Margins:** Fixed distances enforced between components.
- **Opposing Constraints:** Connecting both sides of an axis (e.g., left and right) to create flexibility.
- **Bias:** Adjusting the positioning of an element along an axis when opposing constraints are set (default is 50%).



Chains & Helpers

- **Chains:** Group behavior for multiple composables. Styles include *Spread*, *Spread Inside*, and *Packed*.
- **Guidelines:** Invisible anchor lines (vertical or horizontal) to align content.
- **Barriers:** Dynamic boundaries that move based on the size of referenced elements, preventing overlap.



Implementing Constraints

References

Use createRef() or createRefs() to generate IDs for your composables. These references are used to link components together within the layout scope.

Modifiers

Apply Modifier.constrainAs(ref) to children. Inside the lambda, use linkTo() to define relationships (e.g., top.linkTo(parent.top)).

Key Constraint Features

ConstraintSets

Decouple layout logic from UI code. Define constraints separately and pass them to the ConstraintLayout composable.

Guidelines

Create absolute or percentage-based anchors using createGuidelineFromStart or createGuidelineFromTop.

Barriers

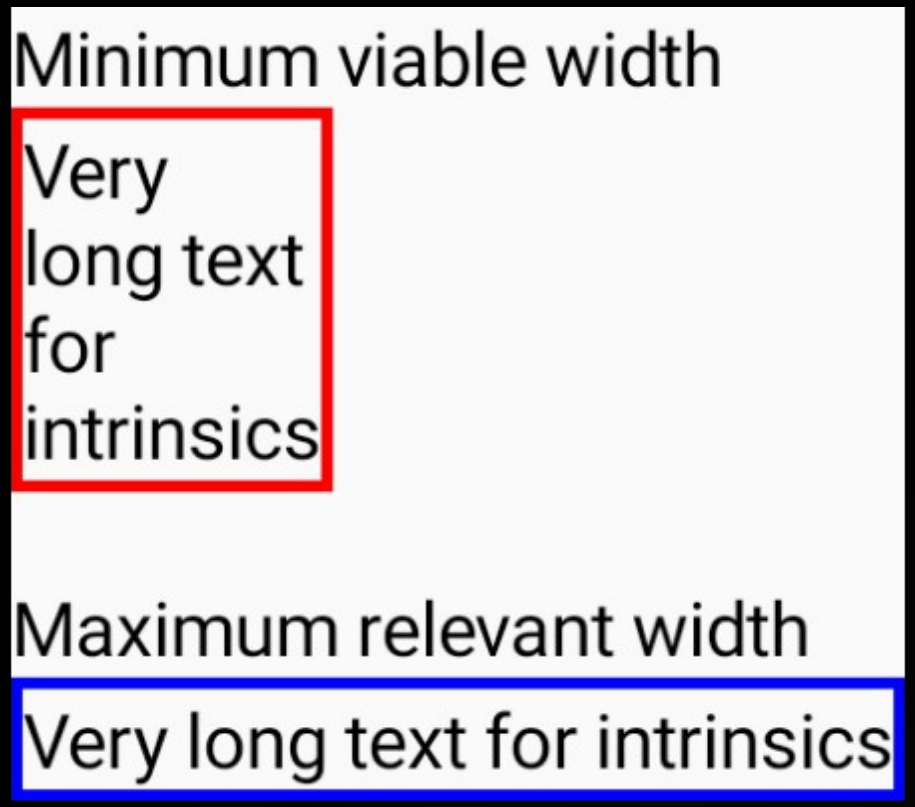
Use createEndBarrier() to create a virtual wall that adapts to the widest element in a set.

Intrinsic Measurements

Solving the "Measure Once" Constraint

The "Measure Once" Rule

- Compose strictly measures elements once for performance.
- **Problem:** A parent container sometimes needs to know the size of its children *before* they are measured (e.g., matching width to the widest child).
- **Solution:** IntrinsicSize.Min or IntrinsicSize.Max allows pre-measurement queries without breaking the single-pass rule.

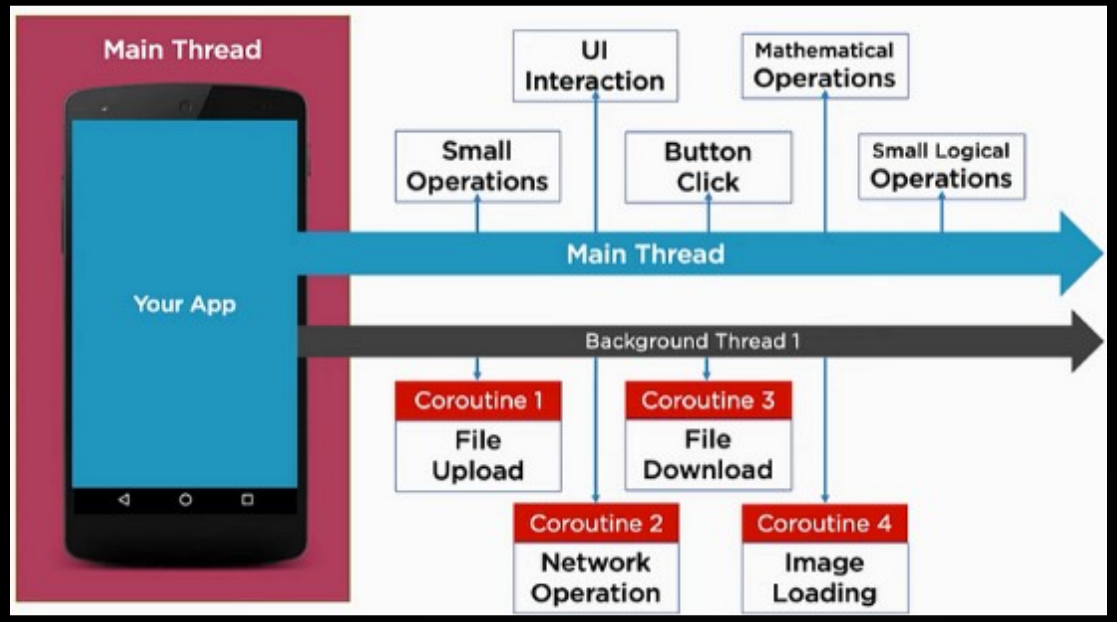


Coroutines & Side Effects

Asynchronous Programming in Compose

Coroutines Basics

- **Main Thread:** Handles UI updates. Blocking it freezes the app.
- **Coroutines:** Lightweight threads for background tasks without blocking the UI.
- **Scope:** Use rememberCoroutineScope() to launch jobs tied to the composable's lifecycle.
- **Dispatchers:** IO for data/network, Main for UI, Default for CPU tasks.



Managing Side Effects

LaunchedEffect

Launches a coroutine when the composable enters the composition. It is cancelled when the composable leaves or when the key parameter changes.

SideEffect

Runs on every successful recomposition. Useful for syncing Compose state with non-Compose code (like updating system UI bars).

Lists & Grids

Handling Large Datasets Efficiently

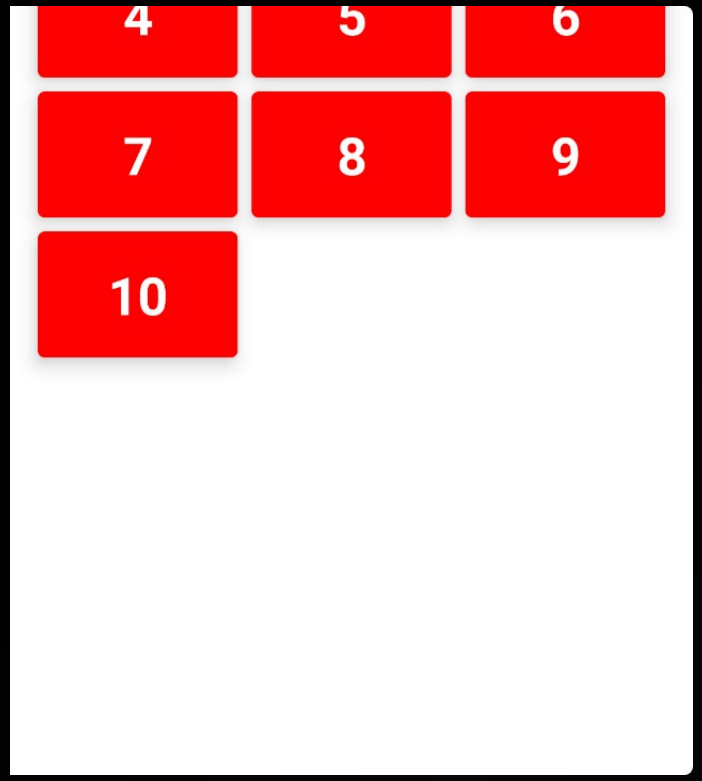
Lazy Lists

- **LazyColumn / LazyRow:** Only render visible items, recycling views as the user scrolls. Essential for performance with large lists.
- **ScrollState:** Use rememberLazyListState() to control scrolling programmatically or observe scroll position.
- **Sticky Headers:** Group items visually using stickyHeader within the list scope.

Lazy Grids

LazyVerticalGrid

- Creates scrollable 2D grids with powerful customization options.
- **Adaptive Cells:** Automatically fits as many columns as possible based on a minimum size (e.g., GridCells.Adaptive(128.dp)).
- **Fixed Cells:** Forces a specific number of columns regardless of screen width (e.g., GridCells.Fixed(3)).



Questions?

Thank you for exploring Advanced Jetpack Compose.